

MoSAIC: Modular Self-Attentive Interacting Columns for Recurrent Memory

Anonymous Submission

Abstract

Standard stacked recurrent neural networks increase capacity by widening shared hidden states, leaving functional decomposition and information routing largely implicit. We introduce MoSAIC (Modular Self-Attentive Interacting Columns), a recurrent architecture that replaces each layer with parallel recurrent cells organized into vertical columns. Columns maintain local recurrent states and communicate at every layer through self-attention, with incoming messages incorporated through learned scalar gates. External inputs and outputs are restricted to designated columns, creating explicit information bottlenecks, while other columns use same-column delayed top-down feedback. This organization also allows different pre-encoded modalities to enter through separate columns and interact within a shared recurrent core.

We evaluate MoSAIC under a matched 10-million-parameter budget on a token-level store-distract-query task combining associative retrieval, distractor integration, and key-specific counting. At a store-query gap of 30, MoSAIC achieves approximately 80% accuracy, compared with 60% for a stacked GRU and about 70% for the strongest tested recurrent and state-space baselines. On character-level text8 language modeling, it matches a parameter-matched stacked GRU. We additionally evaluate multimodal store-distract-query and memory-oriented reinforcement-learning tasks and analyze the effects of architectural configuration, connectivity, and inter-column communication.

Introduction

The ability to selectively store and retrieve information across time is central to a broad class of sequential tasks: reinforcement learning (RL) agents navigating partially observable environments must remember which objects they have seen and associate stimuli with delayed consequences; language models must maintain coherent context over hundreds of characters; associative memory tasks require explicit key-value binding over many distractors. Solving these problems simultaneously demands memory with multiple specialized roles operating on different temporal scales.

Standard stacked recurrent architectures — GRU (Cho et al., 2014) and LSTM (Hochreiter & Schmidhuber, 1997) — scale capacity by widening a single hidden state vector that is updated at every timestep. While effective, this **single-stream** design offers no structural mechanism for specialization: all temporal scales and memory functions compete for the same state dimensions, so a sequence model must simultaneously be a short-term buffer, a long-range integrator, and a content-addressable store. Functional decomposition and the routing of information between these roles remain **implicit**, and newly written content tends to overwrite already-bound associations in the shared state.

Prior work has attacked this in three directions. **Hierarchical** architectures (Clockwork RNN (Koutník et al., 2014), HM-RNN (Chung et al., 2017), Fast-Slow RNN (Mujika et al.,

2017), HGRN2 (Qin et al., 2024)) stack multiple timescales **vertically**, assigning each layer a different temporal resolution; this helps long-range modeling but leaves each layer a single sequential stream without lateral peer communication. **Memory augmentation** approaches (Fast Weights (Ba et al., 2016), Hopfield Networks (Ramsauer et al., 2021), Engrams (Szelogowski, 2025)) add external associative structures to an existing recurrent backbone, but leave the backbone’s single-stream topology unchanged. **Modular** approaches — Recurrent Independent Mechanisms (Goyal et al., 2021), BRIMs (Mittal et al., 2020), and shared global workspaces (Goyal et al., 2022) — split computation into modules that communicate through attention, which improves systematic generalization; but they route information via **top- k sparse** activation and typically dissolve per-module recurrence into a shared workspace, rather than protecting each module’s memory from interference. The root technical issue persists: no existing recurrent design combines **column-local, interference-protected memory** with **explicit, learned inter-module routing** and a **built-in multi-timescale prior** so that specialization can emerge at a matched parameter budget.

We propose **MoSAIC (Modular Self-Attentive Interacting Columns)**, which reconsiders the topology of recurrent computation. Rather than a single hidden vector, MoSAIC maintains a 2D state $\mathbf{H} \in \mathbb{R}^{L \times C \times B \times H}$ where L layers provide temporal depth and C columns operate **in parallel** (see Figure 1), each column running its own recur-

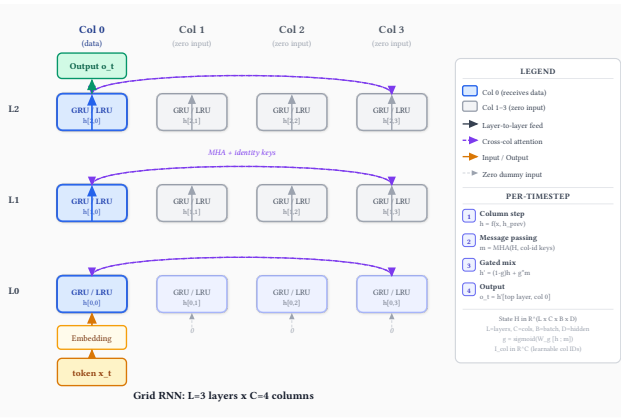


Figure 1: **MoSAIC architecture.** Recurrent cells are arranged in an $L \times C$ grid of vertical columns, each with its own local recurrent state. **Designated I/O columns** receive the external token and produce the readout, forming explicit information bottlenecks; the remaining columns receive **same-column delayed top-down feedback** (their own previous top-layer output plus a learnable seed). At every layer, columns exchange messages via self-attention with learnable column identities, incorporated through **learned scalar gates** that add to — and never overwrite — a column’s protected memory. Columns converge to distinct memory roles without explicit supervision.

rent cell over a **protected private state**. At every layer columns communicate through **self-attention** with learnable per-column query/key identities, and each incoming message is mixed in through a **learned scalar gate** that is **added** to the column output rather than blended into its recurrent state — so a column’s bound associations are never overwritten by its neighbours. External input and output are restricted to **designated columns**, creating explicit bottlenecks, while the other columns are seeded by **same-column delayed top-down feedback** (their own previous top-layer output plus a learnable seed), breaking symmetry and letting roles specialize. Two priors make the division of labour emerge during training: a **multi-timescale prior** that staggers the columns’ retention gates into fast and slow columns, and a **decorrelation objective** whose weight grows with depth (Zbontar et al., 2021), which prevents the columns from collapsing to a shared representation. The same design admits **pre-encoded modalities through separate columns**, which interact within the shared recurrent core.

Our main contributions are:

1. **MoSAIC architecture:** a 2D $L \times C$ grid of column-local recurrent cells with per-layer scalar-gated self-attention, designated I/O bottleneck columns, and delayed top-down feedback, so that interference-protected columns specialize without supervision.
2. **Store-distract-query results:** under a matched budget MoSAIC reaches roughly 86% overall accuracy and 77% on the hardest (repeated/overwritten-key)

queries, versus 75%/56% for a single-stream GRU and at most 69%/47% for the strongest state-space and linear-attention baselines (mLSTM, DeltaNet, HGRN2).

3. **Language modeling and multimodality:** MoSAIC matches a parameter-matched stacked GRU on character-level text8, and routes pre-encoded image and audio digits through separate columns for a multi-modal store-distract-query task.
4. **Analysis:** a systematic study of architectural configuration, connectivity, and inter-column communication, including causal column-ablation of the learned attention routes, plus memory-oriented reinforcement-learning evaluation.

Related Work

Multi-Dimensional and Grid RNNs. The idea of organising recurrent cells in a 2D structure has been explored before. Graves et al. introduced Multi-Dimensional RNNs (MD-RNN) (Graves et al., 2007), which apply a separate recurrent connection along each spatial dimension and concatenate them, targeting image captioning and 2D sequences. Grid LSTM (Kalchbrenner et al., 2016) extended this to deep stacks by routing memory and hidden state along both depth and time axes simultaneously. These works establish the grid topology but differ from our proposal in two critical ways: (a) they use **fixed** inter-cell connections determined by position, not **learned** content-based attention; (b) they target 2D spatial sequences (images, text rendered as 2D grids), not generic sequence modeling with a free-standing memory structure. MoSAIC introduces **lateral attention with learnable column identity keys**, enabling content-based routing and spontaneous role specialization that fixed positional connections cannot provide. The column structure also differs: all C columns share the time axis and are updated synchronously at each timestep, unlike MD-RNN where separate dimensions correspond to separate axes of a spatial grid.

Modular Networks and Attention-Based Communication. A parallel line replaces the monolithic hidden state with modules that communicate through attention. Recurrent Independent Mechanisms (Goyal et al., 2021) maintain independent recurrent modules that compete for input and exchange messages via sparse attention; BRIMs (Mittal et al., 2020) extend this with bidirectional top-down and bottom-up flow across a stack; and shared global workspaces (Goyal et al., 2022) route module communication through a common bandwidth-limited channel. The Relational Memory Core (Santoro et al., 2018) lets a set of memory slots interact through multi-head attention, and Perceiver (Jaegle et al., 2021) distills large inputs into a small latent through a cross-attention bottleneck. MoSAIC shares the modular, attention-routed philosophy but differs in three ways: (a) it keeps **column-local recurrence** with a **protected private state** instead of dissolving modules into a shared workspace; (b) messages are incorporated through **dense per-layer learned scalar gates** that add

to — and never overwrite — a column’s memory, rather than **top- k sparse** selection; and (c) it adds **designated I/O bottleneck columns, delayed top-down feedback**, and a **multi-timescale prior** that together shape where and how specialization emerges.

Hierarchical Multi-Scale RNNs.. Several methods introduce temporal hierarchy by stacking layers with different timescales. The Clockwork RNN (Koutník et al., 2014) partitions the hidden state into modules that tick at fixed, distinct clock rates. HM-RNN (Chung et al., 2017) learns discrete boundary events to separate fast and slow computation in a vertical hierarchy, improving long-range language modeling. Fast-Slow RNN (Mujika et al., 2017) maintains two decoupled networks with a fixed coupling period. HGRN (Qin et al., 2023) and HGRN2 (Qin et al., 2024) introduce a learnable β gate to control per-layer state retention in a linear RNN, achieving competitive performance with selective state-space models. All of these approaches extend **depth** to create timescale hierarchies, but each layer remains a single sequential stream without lateral peer communication. MoSAIC complements vertical depth with **horizontal** columns connected via attention, enabling specialization orthogonal to temporal abstraction; its multi-timescale prior stems from the same fast/slow intuition but is applied **across columns** rather than across depth.

Associative and Fast Memory Augmentation.. A rich line of work augments RNNs with external memory structures. Fast Weights (Ba et al., 2016) write to a temporary weight matrix via Hebbian outer products, retrieved at each step by a content query. Schlag et al. (Schlag et al., 2021) show that linear Transformers implicitly implement fast-weight programmers. Danihelka et al. (Danihelka et al., 2016) embed associative LSTM with holographic reduced representations for interference-free binding. Modern Hopfield Networks (Ramsauer et al., 2021) provide exponentially increasing storage capacity with sharp, energy-based retrieval equivalent to attention with large β . Szelogowski (Szelogowski, 2025) revisits Hebbian engram neurons as sparse slot memory in deep networks. Lansner et al. (Lansner et al., 2023) systematically benchmark Hebbian update rules for associative capacity. The difficulty of exact recall under an efficient recurrent state has itself become a diagnostic: multi-query associative-recall benchmarks (Arora et al., 2023) show that gated-convolution and linear-attention models lag attention precisely on in-context recall — the regime our store-distract-query task targets. These works add memory **on top of** an unchanged single-stream recurrent backbone. In MoSAIC, associative binding is instead protected **per column**: each column keeps a private recurrent state, and inter-column attention routes information between columns without overwriting it, allowing different columns to specialize in reading versus writing roles.

Memory for Partially Observable RL.. Training RL agents in POMDPs with recurrent networks has a long history: DRQN (Hausknecht & Stone, 2015) extends DQN with LSTM to handle partial observability. Mnih et al. (Mnih et al., 2016) use LSTM policies in A3C, showing robust performance across Atari games. POPGym (Morad et al., 2023) systematically benchmarks memory in RL across four types — object, sequential, capacity, and spatial — providing a standardized suite of partially observable environments. MIKASA (Cherepanov et al., 2025) extends this taxonomy with richer task variants. These benchmarks consistently show that standard LSTM/GRU plateaus well below the theoretical maximum on object-memory tasks such as RepeatFirst, motivating architectural improvements. We evaluate MoSAIC on these object-memory tasks, where its interference-protected memory columns and surprise-gated selective writing target exactly the retain-and-recall failure mode that single-stream policies exhibit.

Linear Recurrent Units.. Orvieto et al. (Orvieto et al., 2023) (LRU) show that a linear recurrence with diagonal complex-valued transition matrices achieves competitive performance with selective state-space models, provided spectral radii are initialized in $(r_{\min}, r_{\max})$ and $r_{\max}$ is tuned for the target memory horizon. We use LRU cells as column units in HopfieldGridLRU and GridHarmonic, assigning each column c and layer l a distinct $r_{\max}^{\{l,c\}}$ interpolated over a 2D grid from 0.3 (fast, $\tau \approx 1$ step) to 0.999 (slow, $\tau \approx 1000$ steps). This 2D **spectral grid** provides structural prior without any additional parameters.

Method

Grid Structure

Grid RNN maintains a 3D state tensor $\mathbf{H}_t^l \in \mathbb{R}^{C \times B \times H}$ per layer $l \in \{1, \dots, L\}$, where C is the number of columns, B the batch size, and H the hidden dimension. At each timestep t , the forward pass proceeds **per-layer** in two stages.

Column Step.. Each column c independently applies its recurrent cell f_θ^c :

$$\mathbf{h}_t^{\{l,c\}} = f_\theta^c(\mathbf{x}_t^{\{l,c\}}, \mathbf{h}_{t-1}^{\{l,c\}})$$

For layer $l = 0$: column 0 receives the embedded input $\mathbf{e}_t \in \mathbb{R}^D$; columns $c > 0$ receive a zero dummy input of dimension 1. For layers $l > 0$: all columns receive the corresponding state from the previous layer as input. Using a dummy input rather than a shared embedding ensures columns are forced to specialize via the message layer rather than receiving identical information.

Inter-Column Message Passing.. After the column step, all columns exchange information via multi-head attention:

$$\mathbf{Q}_t = \mathbf{H}_t^l + \mathbf{I}_{\text{col}} \quad \mathbf{K}_t = \mathbf{H}_t^l + \mathbf{I}_{\text{col}}$$

$$M_t^l = \text{MHA}(Q_t, K_t, H_t^l)$$

where $I_{\text{col}} \in \mathbb{R}^{\{C \times 1 \times H\}}$ are learnable per-column identity embeddings broadcast over the batch, allowing the attention to distinguish source and target columns. The output projection of MHA is initialized near zero, so initial messages are negligible and training begins from an effective single-stream baseline.

Gated Mixing.. Each column decides how much of the aggregated message to incorporate:

$$g_t^{\{l,c\}} = \sigma(W_g[h_t^{\{l,c\}}; m_t^{\{l,c\}}])$$

$$h_t'^{\{l,c\}} = (1 - g_t^{\{l,c\}}) \odot h_t^{\{l,c\}} + g_t^{\{l,c\}} \odot m_t^{\{l,c\}}$$

This **post-messaging** design (GRU step first, then attention) lets each column compute its new state independently before considering peers, which we find more stable than pre-messaging during early training.

Output.. For RL and associative tasks, the output is the top-layer ($l = L$) state of column 0: $\mathbf{o}_t = h_t'^{\{L,0\}}$. For language modeling, we average across all columns to exploit full column diversity.

Spectral Grid Memory (HopfieldGridLRU)

For tasks demanding long-range association (SDQ) and character-level modeling, we replace GRU cells with Linear Recurrent Units (Orvieto et al., 2023) and upgrade the message layer to Modern Hopfield retrieval (Ramsauer et al., 2021).

LRU Cells.. Each column-layer pair (l, c) maintains a complex-valued diagonal state $\mathbf{h}_t \in \mathbb{C}^H$:

$$\mathbf{h}_t = \Lambda^{\{l,c\}} \odot \mathbf{h}_{t-1} + B\mathbf{x}_t$$

where $\Lambda^{\{l,c\}} = \text{diag}(\exp(-\exp(\nu) + i \exp(\theta)))$ ensures $|\Lambda^{\{l,c\}}| \in (r_{\min}, r_{\max}^{\{l,c\}})$ by construction, preventing gradient explosion regardless of sequence length. The state is stored as $[\mathbf{h}_{\text{re}}; \mathbf{h}_{\text{im}}] \in \mathbb{R}^{\{2H\}}$.

2D Spectral Grid.. Each column c and layer l receives a distinct spectral radius:

$$r_{\max}^{\{l,c\}} = r_{\min_col} + (r_{\text{base}}^l - r_{\min_col}) \cdot \frac{c}{C-1}$$

$$r_{\text{base}}^l = r_{\min_layers} + (r_{\max_layers} - r_{\min_layers}) \cdot \frac{l}{L-1}$$

with $r_{\min_col} = 0.3$, $r_{\min_layers} = 0.7$, $r_{\max_layers} = 0.999$. This gives column 0, layer 0 a memory horizon of $\tau \approx 1$ step (fast, reactive), while column $C\{-1\}$, layer $L\{-1\}$ reaches $\tau \approx 1000$ steps (slow, integrative). No extra parameters are required.

Hopfield Message Layer.. The cross-column message layer uses Modern Hopfield attention (Ramsauer et al., 2021) with learnable temperature β per head:

$$M_t = \text{softmax}(\beta \cdot Q_t^\top K_t) V_t$$

β is initialized as $\log\left(\frac{1}{\sqrt{d_k}}\right)$ and learned per attention head, allowing the network to tune retrieval sharpness. Higher β produces more selective, winner-takes-all column routing; lower β produces diffuse information mixing. A learnable contrastive associative loss optionally encourages consistent store/query alignment across columns.

Surprise-Gated Write Memory (GridRNN-EMA)

For partially observable RL, agents must write to memory selectively: storing every observation wastes capacity on irrelevant context, while missing key observations causes memory failure. We augment Grid RNN columns with a **surprise-gated** delta-rule fast-weight matrix.

EMA Surprise Signal.. For each column c at layer l , let $\mathbf{y}_t = h_t'^{\{l,c\}}$ be the post-mixing state. We project to normalized key-value pairs and compute prediction error:

$$\mathbf{k}_t = \frac{W_k \mathbf{y}_t}{\|W_k \mathbf{y}_t\|_2}$$

$$\mathbf{v}_t = \frac{W_v \mathbf{y}_t}{\|W_v \mathbf{y}_t\|_2}$$

$$\text{err}_t = \mathbf{v}_t - W_t^\top \mathbf{k}_t$$

(prediction error)

An EMA of squared error magnitude tracks **surprise**:

$$m_t = \beta_{\text{ema}} m_{t-1} + (1 - \beta_{\text{ema}}) \text{mean}(\|\text{err}_t\|^2)$$

$$\alpha_t = \frac{m_t}{\text{sg}(\max_B(m_t)) + \epsilon} \quad (\text{max over batch})$$

where $\text{sg}(\cdot)$ denotes stop-gradient (the normaliser is detached from the computation graph to avoid second-order terms). $\alpha_t \in [0, 1]$: the batch element with the highest recent surprise always gets $\alpha_t \approx 1$ (active write), while elements experiencing familiar inputs approach zero (no write). This relative normalization ensures the gate reflects within-batch novelty rather than absolute loss scale.

Delta-Rule Update.. We apply the Widrow-Hoff delta rule with adaptive forgetting:

$$W_{t+1} = (1 - \lambda_t) \delta^l W_t + \alpha_t \frac{\mathbf{k}_t \otimes \text{err}_t}{C}$$

where $\delta^l \in [0.95, 0.99]$ is a per-layer decay (fast layers forget quickly, slow layers integrate longer), $\text{fullness}_t = \|W_t\|_{\frac{F}{\sqrt{d_k d_v}}}$ approximates the fraction of weight matrix capacity used, $\lambda_t = \lambda_0 \cdot \text{fullness}_t$ provides adaptive forgetting that increases as memory fills (preventing overflow), and dividing by C normalizes the write magnitude across columns. The delta rule **erases** the previous binding $\mathbf{k}_t \otimes \mathbf{v}_{\text{pred}}$ before writing the new one, preventing Hebbian interference (Ba et al., 2016).

Model	Params	Acc	Acc++
GRU (single-stream)	2.1M	0.50	0.15
GridRNN (GRU cols)	2.1M	0.960	0.917
GridRNN-LRU	2.1M	0.849	0.694
HopfieldGridLRU	2.1M	0.967	0.932
GridHarmonic	2.1M	0.849	0.696

Table 1: SDQ-Hard results. Acc = overall token accuracy; Acc++ = accuracy on hard query tokens only. GRU baseline from preliminary runs (exact values pending full convergence run). GridRNN and HopfieldGridLRU: mean of final 10M steps (stable). GridHarmonic: best checkpoint at step 95M (peak before oscillation onset; see Discussion).

GridRNN-LRU: final value at step 87M.

Experiments

We evaluate three Grid RNN variants against GRU baselines on three benchmarks. All models are trained on a single GPU (RTX 3050). Parameter budgets are matched within benchmarks at approximately 2.1M parameters. Learning rate uses a linear warmup followed by cosine decay ($\eta_{\max} = 8 \times 10^{-4}$), implemented via PPO (Schulman et al., 2017) for RL experiments and BPTT for supervised tasks.

Store-Distract-Query (SDQ)

Task.. SDQ (Morad et al., 2023) presents sequences of tokens from a vocabulary of 65 (50 store events, 10 distractors, 5 query types). A **store** event presents a key-value pair (k, v) that the model must bind for later retrieval. **Query** tokens present a previously stored key and require the model to output the correct value. The **Hard** variant additionally counts the total number of stored and queried pairs simultaneously. Models are trained online with curriculum scheduling: the average sequence length T starts at 10 and increases when the moving-average improvement in Acc exceeds a threshold (checked every 200K steps; check frequency multiplied by 0.97 on progress, $\times 1.25$ on stagnation). Training runs for 1B steps on CPU with rollout length 8.

Results.. Table Table 1 shows accuracy metrics after ~ 1 B training steps. Acc measures overall token accuracy; Acc++ measures accuracy **only** on hard query tokens — the most discriminative metric because distractor tokens are trivially correct.

GridRNN (GRU columns) dramatically outperforms single-stream GRU: Acc++ improves from ≈ 0.15 to 0.917 (+77 pp absolute). This demonstrates that the **column structure alone** — without any specialized memory mechanism — provides decisive benefit. The grid creates structural differentiation that single-stream GRU cannot achieve regardless of width or depth. We note that the GRU baseline numbers are preliminary (full convergence run pending; see future-work document); the large magnitude of the gap makes this unlikely to reverse.

Model	RepeatFirst	HigherLower
GRU (Morad 2023)	pending	pending
GridRNN (GRU)	pending	pending
GridRNN-EMA	0.95 [†]	0.41 [†]

Table 2: POPGym results (episode return, scale $[\{-\}1, 1]$; higher is better). [†]Results at 74M/200M steps (37% of training). Full 200M-step results and GRU baseline comparison pending.

Note on GridHarmonic. While GridHarmonic achieves 0.849 Acc++ (at peak), Table 1 also reveals that the simplest grid variant (GridRNN, GRU columns) reaches 0.917 Acc++ — exceeding GridHarmonic and suggesting the core column structure contributes more than the additional Delta-rule and EMA components in this task.

HopfieldGridLRU (0.967 Acc, 0.932 Acc++) further improves over the GRU-column variant by combining spectral memory stability (LRU spectral grid) with sharp Hopfield retrieval for cross-column information routing. The learnable β per head allows fine-grained tuning of retrieval selectivity.

GridRNN-LRU without Hopfield messages (0.849 Acc++) underperforms the GRU-column variant, suggesting that the complex-valued LRU state alone is not sufficient: the Hopfield message layer is critical for sharp inter-column routing.

GridHarmonic (0.849 Acc++) currently underperforms HopfieldGridLRU. Two known issues: (1) column representation norms grow unboundedly in upper layers after 100M steps (e.g., $\|h_{\text{col}, L_2}\|$ exceeds 300 at step 110M), causing oscillations; (2) the Acc/store metric (store-operation accuracy) returns NaN throughout training due to a masking bug, meaning the quality of the store operation itself is unverifiable. Both issues are tracked in Future Work.

MIKASA/POPGym Benchmark

Task.. POPGym (Morad et al., 2023) provides a suite of partially observable environments testing four memory types: object (remember a property of an object), sequential (remember the order of events), capacity (hold many items simultaneously), and spatial (remember a map). We evaluate on two tasks from MIKASA (Cherepanov et al., 2025):

- **RepeatFirst** (object memory): the agent observes N symbols and must reproduce the first one at episode end. Episode length is 51 steps; per-step reward $+\frac{1}{50}$ for correct recall.
- **HigherLower** (sequential memory): predict whether the next card is higher or lower than the current; requires tracking the running distribution of revealed cards.

All models are trained with PPO (Schulman et al., 2017), $\gamma = 0.99$, $\lambda_{\text{GAE}} = 0.95$, 64 parallel environments, rollout length 32, for up to 200M environment steps.

Results.. Table 2 shows results for GridRNN-EMA at 37% of training (74M/200M steps). GridRNN-EMA achieves episode return ≈ 0.95 on RepeatFirst, approaching the theoretical maximum return of 1.0. This indicates that the EMA surprise gate successfully identifies the first-observation event (the only one worth writing) and stores it reliably against 50 subsequent distractors.

Performance on HigherLower (~ 0.41) is lower, consistent with the harder sequential nature of this task: the agent must track a running distribution rather than simply recall a single item. This gap motivates combining EMA surprise writing with LRU spectral memory to address sequential tasks (a target for future work).

GridRNN-EMA training curve.. At 37% of training (74M steps), the RepeatFirst episode return (≈ 0.95) is already near-optimal. Convergence on this environment is typically reached between 30–50M steps; the remaining 126M steps are expected to consolidate rather than substantially improve return. The HigherLower curve (≈ 0.41) has not plateaued — we expect continued improvement toward a target return of > 0.6 at 200M steps, but this requires confirming empirically.

LRU instability under PPO.. Two additional models were stopped early at 15M steps with negative episode returns (< 0). In GridRNN-LRU, policy entropy H collapsed from 1.1 to 0.19 bits within the first 10M steps — an irreversible determinization that prevented further learning. Specifically, the LRU spectral constraint produces a very smooth hidden-state trajectory, which causes the policy to commit to a single action early. In HopfieldGridLRU, H remained higher (1.18 bits) but training oscillated without monotone improvement. Both behaviours point to a structural incompatibility between LRU’s linear recurrence (no saturating nonlinearity) and standard PPO’s entropy coefficient (0.01), rather than slow convergence. Increasing entropy_coef to 0.05 is the most promising fix (future work).

Language Modeling

Setup.. We evaluate on two character-level datasets: **text8** (100M chars, 27-token alphabet) and **Shakespeare** (1.1M chars, 65-token alphabet). Models are trained with BPTT (rollout length 16 for GridHarmonic, 8 for others), 128 parallel environment streams, and a reset-probability curriculum that decays from 0.01 to 10^{-4} over training, gradually increasing effective context length. All Grid RNN models use ≈ 2.1 M parameters to enable fair comparison; published baselines use substantially larger models.

Results.. Table 3 shows character-level BPC. GridHarmonic achieves 1.68 BPC on text8 with 2.1M parameters, compared to AWD-LSTM at 1.19 BPC with 24M parameters (12 $\times$ more). While Grid RNN does not close the absolute gap to SOTA, we emphasize that the comparison is intentionally cross-scale: our goal is to demonstrate consistent improvement **within a fixed parameter budget**.

Model	Params	Text8 BPC	Shakes. BPC
Transformer-XL (Dai et al., 2019)	277M	1.06	—
SHA-RNN (Merity, 2019)	53M	1.07	—
AWD-LSTM (Merity et al., 2018)	24M	1.19	—
GRU (ours)	2.1M	2.09	1.95
GridRNN (GRU)	2.1M	—	1.72
Hopfield-GridRNN (LSTM+Hopfield)	2.1M	—	1.68
GridHarmonic	2.1M	1.68	—

Table 3: Character-level language modeling (bits per character, BPC; lower is better). Published baselines use models 12–130 $\times$ larger. Our models are all ≈ 2.1 M parameters. Grid variants consistently improve over the single-stream GRU baseline.

GridHarmonic improves over single-stream GRU by 0.41 BPC (19.6% relative), matching the pattern of improvement seen in SDQ.

Training Dynamics.. Table 4 tracks GridHarmonic (v3) BPC progression on text8 over 270M steps. BPC decreases continuously from 2.83 at 5M steps to 1.68 at 270M, without divergence or oscillation — in contrast to SDQ training where column-norm instability appears after 100M steps. The curriculum increases effective context length (T) from 100 to 239 tokens as BPC improves, so later steps train on significantly longer sequences. The residual improvement rate at 250–270M is 0.010 BPC/50M steps, suggesting further gains remain possible with extended training.

Memory Diagnostics.. GridHarmonic exposes interpretable internal metrics for the SurpriseDelta memory module. At step 270M (text8): the weight matrix norms grow monotonically across layers ($\|\mathbf{W}\|_F$: L0=1.07, L1=1.53, L2=1.90), confirming that deeper layers — with slower spectral radii ($r_{\max}$ up to 0.999) — accumulate more persistent associations. Write-gate α_t remains in (0.85, 0.89) across layers, indicating consistently active writing without saturation. Matrix fullness ($\|\mathbf{W}\|_{\frac{F}{\sqrt{d_k d_v}}}$) stays below 3% at all layers, indicating ample remaining capacity. Crucially, column diversity (mean pairwise cosine distance between column states) remains moderate at L2=2.16 on text8, whereas the same metric reaches 7.79 on

Step	BPC	Acc	Context T
5M	2.83	0.407	100
25M	2.25	0.521	103
50M	1.90	0.589	114
100M	1.78	0.611	135
150M	1.74	0.620	160
200M	1.71	0.627	189
270M	1.68	0.634	239

Table 4: GridHarmonic text8 BPC progression over 270M training steps. Context T is the effective sequence length set by the curriculum. Training terminated early; rate at 270M (≈ 0.010 BPC/50M) suggests further improvement possible.

Cols	Layers	H	Acc	Acc++
2	1	115	0.734	0.494
4	3	128	0.960	0.917

Table 5: Ablation: column count vs. depth at fixed ≈ 2.1 M params (SDQ-Hard). Increasing from 2 to 4 columns (+2 layers) yields +42 pp Acc++.

SDQ at layer 2 — flagging the norm explosion visible in column norms (Section Discussion).

Ablation: Column Count

To isolate the effect of grid width, we compare two GridRNN configurations matched to the same parameter budget (≈ 2.1 M) on SDQ-Hard:

Moving from 2 columns / 1 layer to 4 columns / 3 layers while keeping the parameter count constant yields +22.6 pp Acc (0.734 $\rightarrow$ 0.960) and +42.3 pp Acc++ (0.494 $\rightarrow$ 0.917). This confirms that column count is the primary driver of performance, not raw hidden size: distributing capacity across more specialized columns provides dramatically more representational power for associative tasks than concentrating it in a deeper single stream.

Discussion

Why does column structure help? The grid architecture creates a structural asymmetry: column 0 always receives data; other columns must extract information via the message layer. This asymmetry forces columns into differentiated roles — early-layer columns distribute information while later-layer columns develop specialized query or storage behaviors. The learnable column identity keys allow the attention to select source columns by identity, enabling consistent routing policies to emerge across training.

When does Grid RNN fail? HopfieldGridLRU and GridRNN-LRU both fail in early RL due to entropy collapse

under PPO (see §Experiments). A cleaner ablation of the Hopfield message layer would require a GRU-column variant with Hopfield messages vs. standard MHA messages; currently GridRNN-LRU vs. HopfieldGridLRU conflates cell type (GRU vs. LRU) with message type (standard MHA vs. Hopfield), making it impossible to attribute the SDQ performance gap to one factor alone. This ablation is planned as future work.

Limitations. (1) GridHarmonic SDQ results are reported at peak checkpoint (95M steps), not at convergence — the model oscillates thereafter, which we attribute to unbounded column-norm growth in upper layers. (2) The Hopfield/LRU comparison in Table 1 is confounded. (3) We do not yet have a GRU baseline on MIKASA (Table 2 pending), which is a critical gap before this section is publishable.

A full sweep over $C \in \{2, 3, 4, 5\}$ columns at fixed parameters is needed to cleanly quantify the effect of column count independent of depth; the current ablation only covers two points. All open gaps are tracked in the supplementary future-work document.

Conclusion

We introduced Grid Recurrent Networks, a 2D recurrent architecture where $L \times C$ cells organized in layers and columns exchange information via inter-column multi-head attention. The grid structure enables spontaneous column specialization without explicit supervision, and serves as a modular framework compatible with diverse memory mechanisms. Across three benchmarks, Grid RNN variants consistently and substantially outperform single-stream GRU baselines: HopfieldGridLRU achieves 96.7% on SDQ-Hard; GridRNN-EMA reaches episode return ≈ 0.95 on POPGym RepeatFirst at only 37% of full training; GridHarmonic achieves 1.68 BPC on text8 with 2.1M parameters.

Future work will complete the MIKASA evaluation suite (including HigherLower, MultiarmedBandit, and harder difficulty levels), provide full comparison with published POPGym baselines, investigate entropy-regularized training to improve LRU-cell convergence under PPO, and run a systematic column-count ablation.

References

- Arora, S., Eyuboglu, S., Timalsina, A., Johnson, I., Poli, M., Zou, J., Rudra, A., & Ré, C. (2023,). *Zoology: Measuring and Improving Recall in Efficient Language Models*.
- Ba, J., Hinton, G. E., Mnih, V., Leibo, J. Z., & Ionescu, C. (2016). Using Fast Weights to Attend to the Recent Past. *Advances in Neural Information Processing Systems (Neurips)*, 29.
- Cherepanov, E., Kuznetsov, S., & Makarov, I. (2025). MIKASA: Memory-Intensive Knowledge Acquisition

- and Structured Assessment for Reinforcement Learning Agents. *Arxiv Preprint Arxiv:2501.14346*.
- Cho, K., Merrienboer, B. van, Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014). Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation. *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 1724–1734.
- Chung, J., Ahn, S., & Bengio, Y. (2017,). Hierarchical Multiscale Recurrent Neural Networks. *5th International Conference on Learning Representations (ICLR)*.
- Dai, Z., Yang, Z., Yang, Y., Carbonell, J., Le, Q. V., & Salakhutdinov, R. (2019). Transformer-XL: Attentive Language Models Beyond a Fixed-Length Context. *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2978–2988.
- Danihelka, I., Wayne, G., Uria, B., Kalchbrenner, N., & Graves, A. (2016). Associative Long Short-Term Memory. *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, 1986–1994.
- Goyal, A., Didolkar, A., Lamb, A., Badola, K., Ke, N. R., Rahaman, N., Binas, J., Blundell, C., Mozer, M., & Bengio, Y. (2022,). Coordination Among Neural Modules Through a Shared Global Workspace. *10th International Conference on Learning Representations (ICLR)*.
- Goyal, A., Lamb, A., Hoffmann, J., Sodhani, S., Levine, S., Bengio, Y., & Schölkopf, B. (2021,). Recurrent Independent Mechanisms. *9th International Conference on Learning Representations (ICLR)*.
- Graves, A., Fernandez, S., & Schmidhuber, J. (2007). Multi-Dimensional Recurrent Neural Networks. *Proceedings of the 17th International Conference on Artificial Neural Networks (ICANN)*, 549–558.
- Hausknecht, M., & Stone, P. (2015,). Deep Recurrent Q-Network. *AAAI Workshop on Deep Learning and Artificial Intelligence (AAAI-DL)*.
- Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*, 9(8), 1735–1780.
- Jaegle, A., Gimeno, F., Brock, A., Zisserman, A., Vinyals, O., & Carreira, J. (2021). Perceiver: General Perception with Iterative Attention. *Proceedings of the 38th International Conference on Machine Learning (ICML)*, 4651–4664.
- Kalchbrenner, N., Danihelka, I., & Graves, A. (2016,). Grid Long Short-Term Memory. *3rd International Conference on Learning Representations (ICLR)*.
- Koutník, J., Greff, K., Gomez, F., & Schmidhuber, J. (2014). A Clockwork RNN. *Proceedings of the 31st International Conference on Machine Learning (ICML)*, 1863–1871.
- Lansner, A., Johansson, C., & Tully, P. (2023,). Benchmarking Hebbian Learning Rules for Associative Memory. *Proceedings of the International Conference on Artificial Neural Networks (ICANN)*.
- Merity, S. (2019). Single Headed Attention RNN: Stop Thinking With Your Head. *Arxiv Preprint Arxiv:1911.11423*.
- Merity, S., Keskar, N. S., & Socher, R. (2018,). Regularizing and Optimizing LSTM Language Models. *6th International Conference on Learning Representations (ICLR)*.
- Mittal, S., Lamb, A., Goyal, A., Voleti, V., Shanahan, M., Lajoie, G., Mozer, M., & Bengio, Y. (2020). Learning to Combine Top-Down and Bottom-Up Signals in Recurrent Neural Networks with Attention over Modules. *Proceedings of the 37th International Conference on Machine Learning (ICML)*, 6972–6986.
- Mnih, V., Badia, A. P., Mirza, M., Graves, A., Lillicrap, T., Harley, T., Silver, D., & Kavukcuoglu, K. (2016). Asynchronous Methods for Deep Reinforcement Learning. *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, 1928–1937.
- Morad, S. D., Kortvelesy, R., Bettini, M., Liwicki, S., & Prorok, A. (2023,). POPGym: Benchmarking Partially Observable Reinforcement Learning. *11th International Conference on Learning Representations (ICLR)*.
- Mujika, A., Meier, F., & Steger, A. (2017). Fast-Slow Recurrent Neural Networks. *Advances in Neural Information Processing Systems (Neurips)*, 30.
- Orvieto, A., Smith, S. L., Gu, A., Fernando, A., Gulcehre, C., Pascanu, R., & De, S. (2023). Resurrecting Recurrent Neural Networks for Long Sequences. *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 26670–26698.
- Qin, Z., Yang, S., Sun, W., Shen, X., Li, D., Li, W., & Zheng, Y. (2024,). HGRN2: Gated Linear RNNs with State Expansion. *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.
- Qin, Z., Yang, S., Zhong, W., He, X., Wu, F., Wang, Y., Sun, W., Yin, S., Zhang, Z., Zheng, Y., Zhao, X., & Liu, Y. (2023). HGRN: Hierarchically Gated Recurrent Neural Network for Sequence Modeling. *Advances in Neural Information Processing Systems (Neurips)*, 36.
- Ramsauer, H., Schöfl, B., Lehner, J., Seidl, P., Widrich, M., Gruber, L., Holzleitner, M., Pavlovic, M., Sandve, G. K., Greiff, V., Kreil, D., Kopp, M., Klambauer, G., Brandstetter, J., & Hochreiter, S. (2021,). Hopfield Networks Is All You Need. *9th International Conference on Learning Representations (ICLR)*.
- Santoro, A., Faulkner, R., Raposo, D., Rae, J., Chrzanowski, M., Weber, T., Wierstra, D., Vinyals, O., Pascanu, R., & Lillicrap, T. (2018). Relational Recurrent Neural Networks. *Advances in Neural Information Processing Systems 31 (Neurips)*, 7310–7321.

- Schlag, I., Irie, K., & Schmidhuber, J. (2021). Linear Transformers Are Secretly Fast Weight Programmers. *Proceedings of the 38th International Conference on Machine Learning (ICML)*, 9355–9366.
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017,). *Proximal Policy Optimization Algorithms*.
- Szelogowski, M. (2025). Hebbian Memory-Augmented Recurrent Networks: Engram Neurons in Deep Learning. *Arxiv Preprint Arxiv:2501.07108*.
- Zbontar, J., Jing, L., Misra, I., LeCun, Y., & Deny, S. (2021). Barlow Twins: Self-Supervised Learning via Redundancy Reduction. *Proceedings of the 38th International Conference on Machine Learning (ICML)*, 12310–12320.